DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2014-2015

ANALYSIS OF ALGORITHM LABORATORY

Class: Second Year **Sem: IV**

INDEX

S. No.	Practical	Page No.
1	Study and implement selection sort algorithms.	2
2	Study and implement Recursive Binary Search by using divide and conquer strategy.	6
3	Study and implement Merge Sort Algorithm by using divide and conquer strategy.	12
4	Study and implement Quick Sort Algorithm by using divide and conquer strategy.	18
5	Study and implement Dijkstra's algorithm for finding shortest path between two nodes in given graph by using greedy strategy.	21
6	Study and implement Prim's Algorithm for finding minimum cost of spanning tree of given graph.	26
7	Study and implement optimal solution of Knapsack Problem by using Greedy Method.	30
8	Study and implement optimal solution of 0/1 knapsack problem using dynamic programming.	34
9	Study Eight Queens problem and solve by using Back Tracking Technique.	37
10	Study and implement travelling salesperson problem using dynamic programming.	42

PREPARED BY APPROVED BY

EXPERIMENT NO:1 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: SELECTION SORT

AIM: To study and implement selection sort algorithm.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

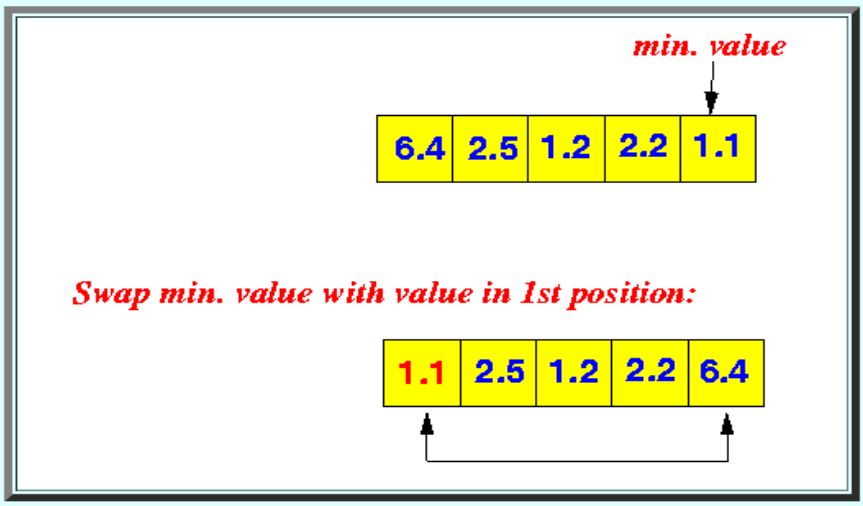
b) Theory

Selection sort is a sorting algorithm, specifically an in-place comparison sort. Selection sort is noted for its simplicity, and it has performance advantages over more complicated algorithms in certain situations, particularly where auxiliary memory is limited.

The algorithm divides the input list into two parts: the sub list of items already sorted, which is built up from left to right at the front (left) of the list, and the sub list of items remaining to be sorted that occupy the rest of the list. Initially, the sorted sub list is empty and the unsorted sub list is the entire input list. The algorithm proceeds by finding the smallest (or largest, depending on sorting order) element in the unsorted sub list, exchanging it with the leftmost unsorted element (putting it in sorted order), and moving the sublist boundaries one element to the right.

- Find the **minimum value** in the list of number
- **Swap** the **minimum value** with the **value in the first position**

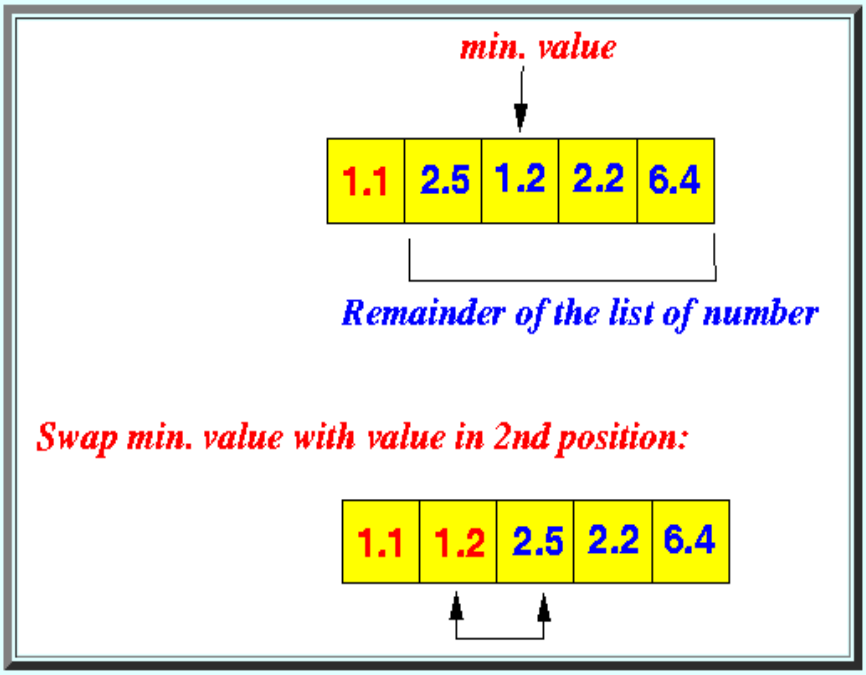
Illustration:



We know that the *first* **element** is in its **rightful (correct) position**.

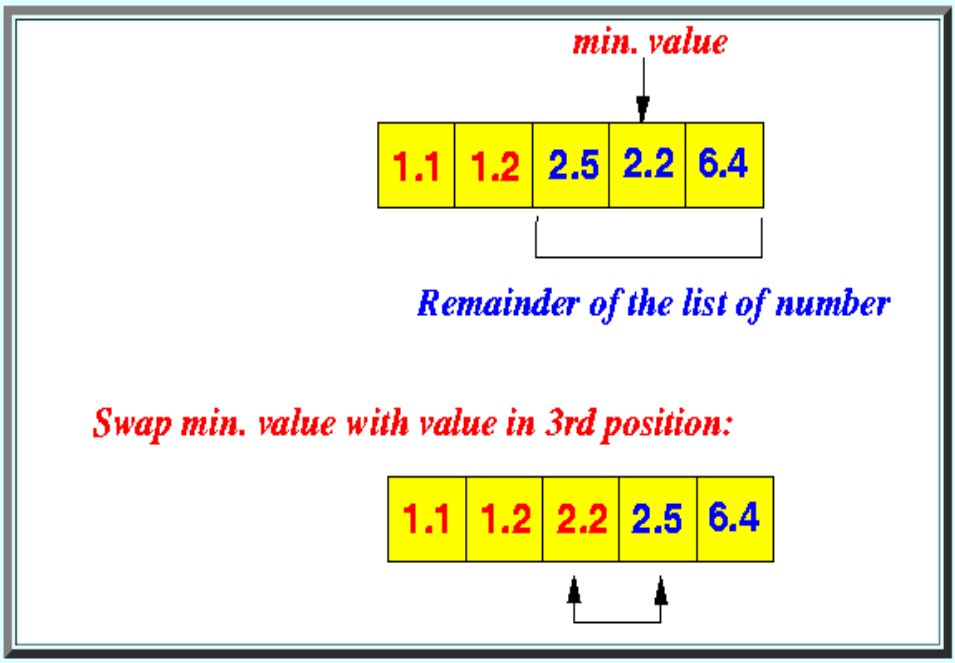
- **Repeat the steps above** for the **remainder of the list** (**starting** at the *second position* and **advancing each time**)

Step 2:



We know that the *first* and *second* elements are in their rightful (correct) position.

Step 3:



Now we are certain that the first 3 numbers are in the correct order.

And so on...

c) Algorithm

Step 1: The list is divided into two sub-lists, sorted and unsorted, which are divided by an imaginary wall.

Step 2: We find the smallest element from the unsorted sub-list and swap it with the element at the beginning of the unsorted data.

Step 3: After each selection and swapping, the imaginary wall between the two sub-lists move one element ahead, increasing the number of sorted elements and decreasing the number of unsorted ones.

Step 4: Each time we move one element from the unsorted sub-list to the sorted sub-list, we say that we have completed a sort pass.

Step 5: A list of n elements requires n-1 passes to completely rearrange the data.

d) Program

```
#include<stdio.h>
int main(){

 int s,i,j,temp,a[20];

 printf("Enter total elements: ");
 scanf("%d",&s);

 printf("Enter %d elements: ",s);
 for(i=0;i<s;i++)
    scanf("%d",&a[i]);

 for(i=0;i<s;i++){
    for(j=i+1;j<s;j++){
        if(a[i]>a[j]){
           temp=a[i];
          a[i]=a[j];
          a[j]=temp;
        }
    }
 }

 printf("After sorting, list is: ");
 for(i=0;i<s;i++)
    printf(" %d",a[i]);

 getch();
}
```

e) Output

```
Enter total elements:     5
Enter 5 elements:
                          5
                          8
                          2
                          7
                          4
After sorting, list is:
                2   4     5     7     8
```

f) Conclusion

Thus selection sort is studied and implemented successfully.

VIVA QUESTIONS

EXPERIMENT NO:2 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: BINARY SEARCH

AIM: To study and implement Recursive Binary Search by using divide and conquer strategy.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory

A binary search tree is a binary tree. It may be empty. If it is not empty, then it satisfies the following properties:

1. Every element has a key and no two elements have the same key (i.e. the keys are distinct).
2. The keys (if any) in the left subtree are smaller than the key in the root.
3. The keys (if any) in the right subtree are larger than the key in the root.
4. The left and right subtree is a also binary search tree.

Let a_i $1 <= i <= n$, be a list of elements that are sorted in non-decreasing order. Consider the problem of determining whether a given element x is present in the list. If x is present, we are to determine a value j such that $a_j = x$. If x is not in the list, then j is to be set to zero.

EXAMPLE: RECURSIVE BINARY SEARCH (SUCCESSFUL SEARCH)

We begin by comparing the middle element of the array with the search key. If they are equal, we found the search key and return the index of the middle element. This is a base case.

STEP 1. We will search for the value 7 in the below given sorted array:

Value	3	6	7	8	12	15	22	36	**45**	48	51	53	64	69	72	89	95
Index	0	1	2	3	4	5	6	7	**8**	9	10	11	12	13	14	15	16

To begin, we find the index of the center element, which is 8, and we compare our search key (7) with the value 45.

STEP 2. A recursive call is made to search the left subarray, comprised of the array elements with indexes between 0 and 7 included.

Value	3	6	7	**8**	12	15	22	36	45	48	51	53	64	69	72	89	95

Index	0	1	2	**3**	4	5	6	7	8	9	10	11	12	13	14	15	16

The index of the center element is now 3, so we compare 7 to the value 8.

STEP 3. A recursive call is made to search the left subarray, comprised of the array elements with indexes between 0 and 2 included.

Value	3	**6**	7	8	12	15	22	36	45	48	51	53	64	69	72	89	95
Index	0	**1**	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

The index of the center element is now 1, so we compare 7 to the value 6.

STEP 4. A recursive call is made to search the right subarray, comprised of the only array element with index between 2 and 2 included.

Value	3	6	**7**	8	12	15	22	36	45	48	51	53	64	69	72	89	95
Index	0	1	**2**	3	4	5	6	7	8	9	10	11	12	13	14	15	16

The value of the element at index 2 matches the search key, so we have reached a base case and we return the index 2 (successful search).

EXAMPLE: RECURSIVE BINARY SEARCH (VALUE NOT FOUND)

STEP 1. This time, we search for a value not found in the array, 34. Again, we start with the entire array and find the index of the middle element, which is 8.

Value	3	6	7	8	12	15	22	36	**45**	48	51	53	64	69	72	89	95
Index	0	1	2	3	4	5	6	7	**8**	9	10	11	12	13	14	15	16

We compare our search key (34) with the value 45.

STEP 2. A recursive call is made to search the left subarray, comprised of the array elements with indexes between 0 and 7 included.

Value	3	6	7	**8**	12	15	22	36	45	48	51	53	64	69	72	89	95
Index	0	1	2	**3**	4	5	6	7	8	9	10	11	12	13	14	15	16

The index of the center element is now 3, so we compare 34 to the value 8.

STEP 3. A recursive call is made to search the right subarray, comprised of the array elements with indexes between 4 and 7 included.

Value	3	6	7	8	12	**15**	22	36	45	48	51	53	64	69	72	89	95
Index	0	1	2	3	4	**5**	6	7	8	9	10	11	12	13	14	15	16

The index of the center element is now 5, so we compare 34 to the value 15.

STEP 4. A recursive call is made to search the right subarray, comprised of the only array element with index between 6 and 7 included.

Value	3	6	7	8	12	15	**22**	36	45	48	51	53	64	69	72	89	95
Index	0	1	2	3	4	5	**6**	7	8	9	10	11	12	13	14	15	16

The index of the center element is now 6, so we compare 34 to the value 22

STEP 5. A recursive call is made to search the right subarray, comprised of the only array element with index between 7 and 7 included.

Value	3	6	7	8	12	15	22	**36**	45	48	51	53	64	69	72	89	95
Index	0	1	2	3	4	5	6	**7**	8	9	10	11	12	13	14	15	16

The index of the center element is now 7, so we compare 34 to the value 36.
A recursive call is made to search the left subarray, but that left subarray is empty. We have reached the other base case, and we return –1, indicating an unsuccessful search.

c) Algorithm

STEP 1: Start the program.
STEP 2: Read the value of n.
STEP 3: Set a for loop to read the elements of array.
for(i = 0; i< n; i++)
STEP 4: Set a for loop.
for(i = 0; i< n; i++)
STEP 5: Nest another for loop.
for(j = i + 1; j < n; j++)
STEP 6: Check the condition a[i] > a[j].
STEP 7: If so swap the two values using temporary variable t as
t = a[i]
a[i] = a[j]
a[j] = t
STEP 8: Else go back to step 6.
STEP 9: Set a for loop to print the value of array a.
for(i = 0; i< n; i++)

STEP 10: Read the search key as k.

STEP 11: Assign low = 0 and high = n – 1.
STEP 12: Call the function binsearch(a, k, low, high)
STEP 13: Check if ans is not equal to 1, if so print the position b + i. Else print that element is not found.
STEP 14: Stop the program.

FUNCTION BINARY SEARCH (int *x[ ], int x, int low, int high)

STEP 1: Set a while loop till low is greater than high.
STEP 2: Assign mean value of low and high to mid.
mid = (high + low) / 2
STEP 3: Assign the value of x[mid] to p.
p = x[mid]
STEP 4: Check if x < p if so assign.
high = mid - 1
STEP 5: Else check whether x > p if so then assign.
low = mid + 1
STEP 6: Else check whether x == p, if so return mid.
STEP 7: Else return -1.

d) Program

```
#include<stdio.h>
int x;
int main(){

   int a[10],i,j,n,m,c,l,u;

   printf("Enter the number of elements: ");
   scanf("%d",&n);

   printf("Enter the array elements one by one: " );
   for(i=0;i<n;i++){
      scanf("%d",&a[i]);
   }
   printf(" \n Sorted array \n ");
   for(i= 0; i< n; i++)
        for(j = i + 1; j < n; j++)
             if(a[i] > a[j])
             {
                int t = a[i];
                a[i] = a[j];
                a[j] = t;
             }
   for(i =0; i< n; i++)
   printf(" \t %d ", a[i]);
```

```
        printf("\n Enter the number to be search: ");
        scanf("%d",&m);

        l=0,u=n-1;
        c=binary(a,n,m,l,u);
        if(c==0)
            printf("Number is not found.");
        else
            printf("Number is found at position %d",x+1);

        getch();
      }

    int binary(int a[],int n,int m,int l,int u){

        int mid,c=0;

        if(l<=u){
            mid=(l+u)/2;
            if(m==a[mid]){
                c=1;
                x=mid;
            }
            else if(m<a[mid]){
                return binary(a,n,m,l,mid-1);
            }
            else
                return binary(a,n,m,mid+1,u);
        }
        else
          return c;
  }
```

e) Output:

Enter the number of elements: 6
Enter the array elements one by one:
23
45
89
98
09
65

Sorted Array 09 23 45 65 89 98

Enter the element to search 23.
Number is found at position 2.

Enter the element to search 50.
Number not found.

f) Conclusion:

Thus Recursive Binary Search is studied and implemented by using divide and conquer strategy.

VIVA QUESTIONS

EXPERIMENT NO:3	**DOP:**	**DOS:**	**GRADE:**

TITLE OF EXPERIMENT: MERGE SORT

AIM: To study and implement Merge Sort Algorithm by using divide and conquer strategy.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

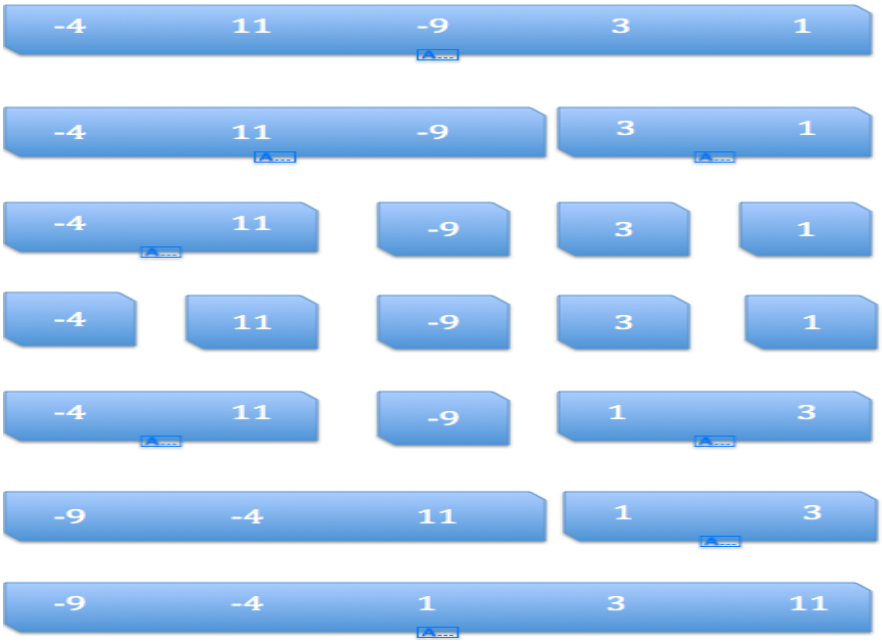
b) Theory

The merge sort algorithm closely follows the divide-and-conquer paradigm. Intuitively, it operates as follows:

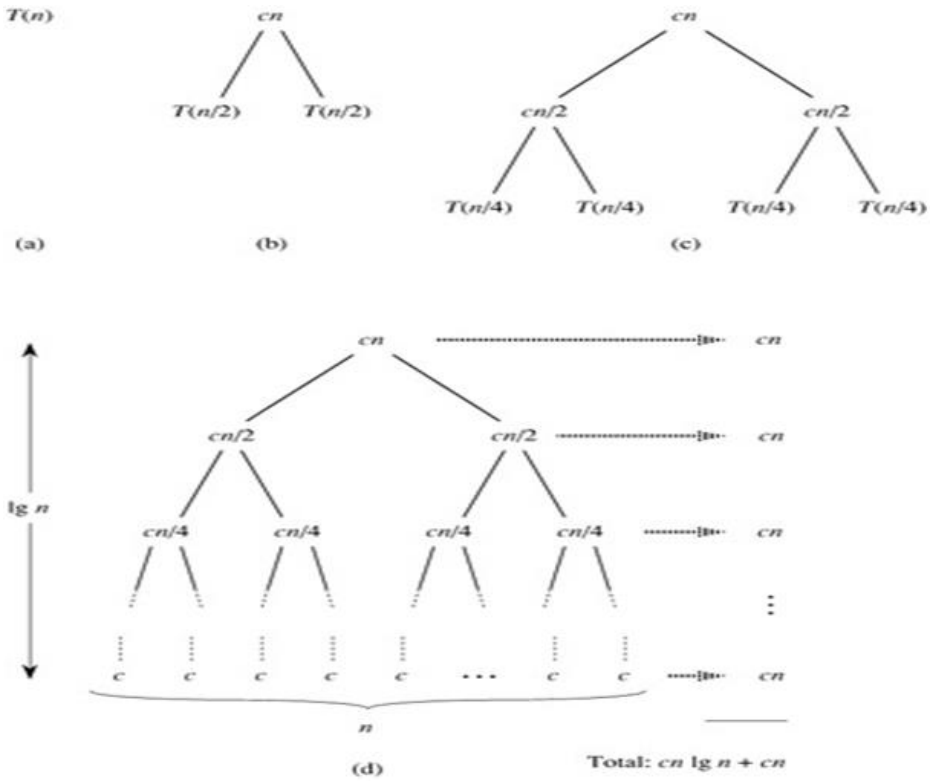
- **Divide:** Divide the n-element sequence to be sorted into two subsequences of n=2 elements each.
- **Conquer:** Sort the two subsequences recursively using merge sort.
- **Combine:** Merge the two sorted subsequences to produce the sorted answer. During the Mergesort process the object in the collection are divided into two collections. To split a collection, Mergesort will take the middle of the collection and split the collection into its left and its right part. The resulting collections are again recursively sorted via the Mergesort algorithm.

Once the sorting process of the two collections is finished, the result of the two collections is combined. To combine both collections Mergesort start at each collection at the beginning. It pick the object which is smaller and inserts this object into the new collection. For this collection it now selects the next elements and selects the smaller element from both collection. Once all elements from both collections have been inserted in the new collection, Mergesort has successfully sorted the collection. To avoid the creation of too many collections, typically one new collection is created and the left and right side are treated as different collections.

Example:



Analysis:



Merge sort complexity in best case=O (n*log (n))

Merge sort complexity in average case=O (n*log (n))

Merge sort complexity in worst case=O (n*log (n))

c) Algorithm:

MERGE (A, p, q, r)

1. n1 =q -p + 1
2. n2 = r - q
3. let L[1.. n1 + 1] and R[1.. n2 + 1] be new arrays
4. for i =1 to n1
5. L[i]=A[p + i – 1]
6. for j =1 to n2
7. R[j]=A[q+j]
8. L[n1 +1]=∞
9. R[n2 +1]=∞
10. i = 1
11. j = 1
12. for k = p to r
13. if L[i]<=R[j]
14. A[k]=L[i]
15. i = i +1
16. else A[k]=R[j]
17. j = j + 1

MERGE-SORT (A, p, r)

1. if p < r
2. q = [(p +r)/2]
3. MERGE-SORT(A, p, q)
4. MERGE-SORT(A, q + 1, r)
5. MERGE(A, p, q, r)

d) Program

```
#include <stdio.h>

void merge(int [], int, int, int);

void mergeSort(int [],int, int);

 int main()

{

   int list[50];

   int i, size;

    printf("Enter total number of elements:");

   scanf("%d", &size);

   printf("Enter the elements:\n");

   for(i = 0; i < size; i++)

   {
```

```
        scanf("%d", &list[i]);
    }
    mergeSort(list, 0, size - 1);
    printf("After merge sort:\n");
    for(i = 0;i < size; i++)
    {
        printf("%d   ",list[i]);
    }
    getch();
}

void mergeSort(int list[],int low,int high)
{
    int mid;
     if(low < high)
    {
        mid = (low + high) / 2;
        mergeSort(list, low, mid);
        mergeSort(list, mid + 1, high);
        merge(list, low, mid, high);
    }
}

void merge(int list[],int low,int mid,int high)
{
    int i, mi, k, lo, temp[50];
    lo = low;
    i = low;
    mi = mid + 1;
    while ((lo <= mid) && (mi <= high))
    {
        if (list[lo] <= list[mi])
```

```
            {
                temp[i] = list[lo];
                lo++;
            }
            else
            {
                temp[i] = list[mi];
                mi++;
            }
            i++;
        }
        if (lo > mid)
        {
            for (k = mi; k <= high; k++)
            {
                temp[i] = list[k];
                i++;
            }
        }
        else
        {
            for (k = lo; k <= mid; k++)
            {
                temp[i] = list[k];
                i++;
            }
        }
        for (k = low; k <= high; k++)
        {
            list[k] = temp[k];
        }
    }
```

e) Output:

Enter total number of elements: 6

Enter the elements: 4
5
3
7
2
9

After merge sort:

2 3 4 5 7 9

f) Conclusion:

Thus Merge Sort Algorithm is studied and implemented successfully by using divide and conquer strategy.

VIVA QUESTIONS

EXPERIMENT NO:4	**DOP:**	**DOS:**	**GRADE:**
TITLE OF EXPERIMENT: QUICK SORT			
AIM: To study and implement Quick Sort Algorithm by using divide and conquer strategy.			

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory

Partition-exchange sort or quicksort algorithm was developed in 1960 by Tony Hoare. He developed the algorithm in order to sort the words to be translated, to make them more easily matched to an already-sorted Russian-to-English dictionary that was stored on magnetic tape.

Quick sort algorithm on average, makes O(n log n) comparisons to sort n items. In the worst case, it makes O(n^2) comparisons, though this behavior is rare. Quicksort is often faster in practice than other O(n log n) algorithms. Additionally, quicksort's sequential and localized memory references work well with a cache. Quicksort is a comparison sort and, in efficient implementations, is not a stable sort. Quicksort can be implemented with an in-place partitioning algorithm, so the entire sort can be done with only O(log n) additional space used by the stack during the recursion.

Quicksort is a divide and conquer algorithm. Quicksort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quicksort can then recursively sort the sub-lists.

1. Elements less than pivot element.
2. Pivot element.
3. Elements greater than pivot element.

Where pivot as middle element of large list. Let's understand through example:

List : 3 7 8 5 2 1 9 5 4

In above list assume **4** is pivot element so rewrite list as:

3 1 2 4 5 8 9 5 7

Here, I want to say that we set the **pivot element** (**4**) which has in left side elements are less than and right hand side elements are greater than. Now you think, how's arrange the less than and greater than elements? Be patient, you get answer soon.

c) Algorithm

Now let's start understand the concept of quick sort. The **steps are**:

1. Pick a pivot element.
2. Reorder the list so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called

the partition operation.

3. Recursively sorts the sub-list of lesser elements and the sub-list of greater elements. The base case of the recursion is lists of size zero or one, which never need to be sorted.

d) Program

```
#include<stdio.h>
void quicksort(int [10],int,int);

int main(){
  int x[20],size,i;

  printf("Enter size of the array: ");
  scanf("%d",&size);

  printf("Enter %d elements: ",size);
  for(i=0;i<size;i++)
    scanf("%d",&x[i]);

  quicksort(x,0,size-1);
  printf("Sorted elements: ");
  for(i=0;i<size;i++)
    printf(" %d",x[i]);
  getch();
}

void quicksort(int x[10],int first,int last){
    int pivot,j,temp,i;

     if(first<last){
        pivot=first;
        i=first;
        j=last;
        while(i<j){
           while(x[i]<=x[pivot]&&i<last)
              i++;
           while(x[j]>x[pivot])
              j--;
           if(i<j){
              temp=x[i];
               x[i]=x[j];
```

```
                x[j]=temp;
            }
        }

        temp=x[pivot];
        x[pivot]=x[j];
        x[j]=temp;
        quicksort(x,first,j-1);
        quicksort(x,j+1,last);
    }
}
```

e) Output:

Enter size of the array: 5
Enter 5 elements:
3 8 0 1 2
Sorted elements:
0 1 2 3 8

f) Conclusion:

Thus Quick Sort Algorithm is studied and implemented successfully by using divide and conquer strategy.

VIVA QUESTIONS

EXPERIMENT NO:5 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: DIJKSTRA'S ALGORITHM

AIM: To study and implement Dijkstra's algorithm for finding shortest path between two nodes in given graph by using greedy strategy.

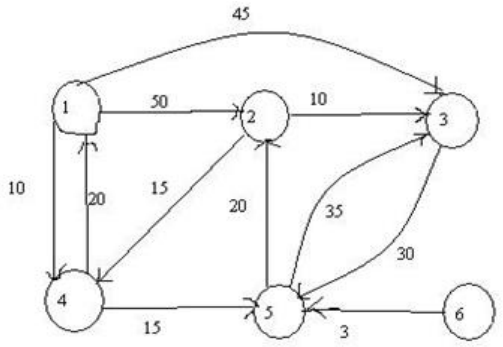
FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory

Let G (V, E) be a graph , V-Vertices & E-Edges .We have to choose one vertex as source , the problem aim is to find out minimum distance between source node and all remaining nodes. This problem is the case of ordered paradigm in Greedy method.



This problem can be implemented by an algorithm called Dijkastra's algorithm.

PATH	COST
1,4	10
1,4,5	25
1,4,5,2	45
1,3	45

c) Algorithm

```
//G be a graph
//Cost matrix [1:n,1:n] for the graph G

//S={set of vertices that path already generated} //Let V be
source vertex

//dist[j];1<=j<=n denotes distance between V and j void main()
{

for i:=1 to n do
        {

                s[i]=false; // initialize s with n dist[i]=cost[v,i];
            //define distance
        }

        s[v]=true; //put v in s

        dist[v]=0.0; //Distance between v and v is 0 for num:=2 to
    n-1 do
        {

                paths from v //choose u from among those vertices not in S such that  ist[
            ]=min;
                s[u]=true;
                for(each w adjascent to u with s[w]=false)
                        if(dist[w]>dist[u]+cost[u,w])
                        then
                                dist[w]=dist[u]+cost[u,w]; //update the distance

        }
}
```

d) Program

```
#include<stdio.h>
#include<conio.h>
#define infinity 32767
int cost[20][20],n,dist[20],s[20],a[20][20];
void setdata();
void getdata();
void path(int);

void setdata()
{
```

```
int i,j,k;
printf("\nEnter number of nodes: "); scanf("%d",&n);
printf("Enter Adjacency Matrix: "); for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
scanf("%d",&a[i][j]);
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
{
 if(i==j)
cost[i][i]=0; else if (a[i][j]!=0)
{
printf("\nEnter cost from %d to %d: ",i,j); scanf("%d",&cost[i][j]);
}
else
cost[i][j]=infinity;
}
}
void getdata()
{
        int i;
        for(i=1;i<=n;i++)
                if(dist[i]==32767)
                        printf("not reachable");
                else
                        printf(" %d",dist[i]);
}

void path(int v)
{
        int i,j,min,u;
        for(i=1;i<=n;i++)
        {
                s[i]=0;
                dist[i]=cost[v][i];
        }
        s[v]=1;
        dist[v]=0;
        for(i=2;i<=n;i++)
        {
                min=32767;
                for(j=1;j<=n;j++)
                if(s[j]==0 && dist[j]<min) u=j;
                s[u]=1;
                for(j=1;j<=n;j++)
                if(s[j]==0 && a[u][j]==1) if(dist[j]>dist[u]+cost[u][j])
                dist[j]=dist[u]+cost[u][j];
```

```
        }
    }


    main()
    {
        int v;
        setdata();
        printf("\nEnter the source vertex: "); scanf("%d",&v);
        path(v);
        printf("\nShortest paths " );
        getdata();
        getch();
    }
```

e) Output:

Enter number of nodes: 6

Enter Adjacency Matrix:

0 1 1 1 0 0
0 0 1 1 0 0
0 0 0 0 1 0
1 0 0 0 1 0
0 1 1 0 0 0
0 0 0 0 1 0

Enter cost from 1 to 2: 50

Enter cost from 1 to 3: 45

Enter cost from 1 to 4: 10

Enter cost from 2 to 3: 10

Enter cost from 2 to 4: 15

Enter cost from 3 to 5: 30

Enter cost from 4 to 1: 20

Enter cost from 4 to 5: 15

Enter cost from 5 to 2: 20

Enter cost from 5 to 3: 35

Enter cost from 6 to 5: 3

Enter the source vertex: 1

Shortest paths:	0	45	45	10	25	not reachable

f) Conclusion:

Thus the Dijkstra's algorithm for finding shortest path between two nodes using greedy strategy has studied and implemented successfully.

VIVA QUESTIONS

EXPERIMENT NO:6 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: PRIM'S ALGORITHM

AIM: To study and implement Prim's Algorithm for finding minimum cost of spanning tree of given graph.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

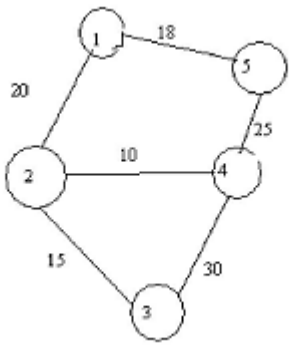
b) Theory

Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph. This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized.

Let G(V,E) be an undirected graph.
A sub-graph $T=(V,E^1)$ is said to be a spanning tree of G if T is a tree.
A Minimum cost spanning tree is a spanning tree with minimum weight.

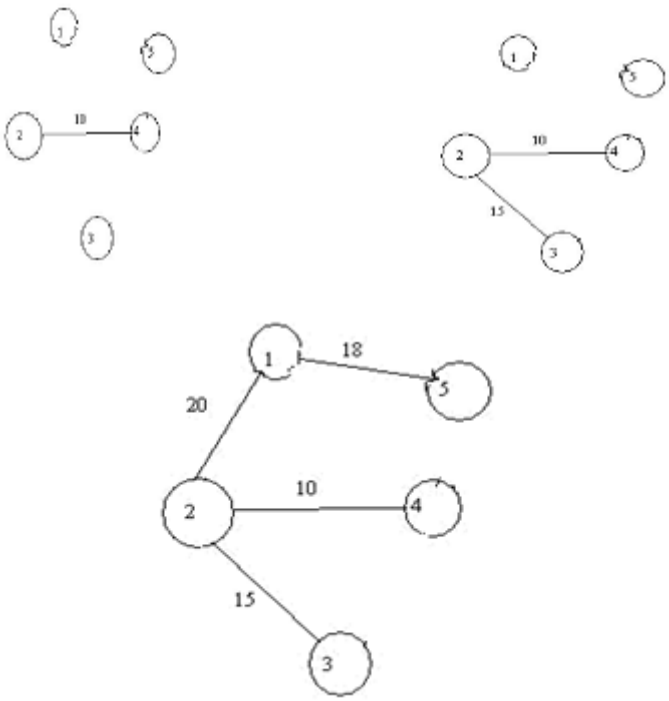
Prim's method:
The Greedy method to obtain MCST builds the tree edge by edge. Choose the edges with minimum cost. Cycles should be avoided.

Graph:



TOTAL COST:118

Stages are:



TOTAL COST :63

c) Algorithm

```
Prim(E,cost,n,t)
//E is the set of edges in G.cost [1:n,1:n] is the cost
//adjacency matrix of an n vertex graph such that cost[i,j] is
//either a positive real number or infinity if no edge of   (i,j) exists
//a minimum spanning tree is computed and stored as a set of

//edges in the array t[1:n-1,1:2] . (t[i,1],t[i,2]  is an ed  e in the minimum cost spanning
//tree .The final cost is returned.
{
                Let (k,l) be an edge of minimum cost in Ei
mincost :=cost[k,l];
t[1,1]:=k;t[1,2]:=l;
for i:=1 to n do //initialize near
if(cost[i,l]<cost[i,k]) th n n ar[i] :=l;
Else
near[i]:=k;
near[k]:=near [l]:=0;
for i:=2 to n-1 do
{
// Find n-2 additional edges for t
Let j be an index such that near[j]!=0and
Cost[j,near[j]] is minimum
                 t[i,1]:=j;t[i,2]:=near[j];
                mincost :=mincost+cost[j,near[j]];
                near[j]:=0;
```

```
for k:=1 to n do //Update near[].
        if((near[k]!=0) and (cost[k,near[k]]>cost[k,j]))
                then near[k]:=j;
        }
return mincost;
}
```

d) Program

```
#include<stdio.h>
#include<conio.h>

int g[20][20],d[20],visited[20],p[20]; int v,e;
int i,j;
void creategraph();
void prim();

main()
{
        creategraph();
        prim();
        getch();
}
void creategraph()
{
        int a,b,w;
        printf("Enter number of vertices:");
        scanf("%d",&v);
        printf("Enter number of edges:");
        scanf("%d",&e);
        for(i=1;i<=v;i++)
                for(j=1;j<=v;j++)
                        g[i][j]=0;
        for(i=1;i<=v;i++)
                p[i]=0;
        visited[i]=0;
        d[i]=32767;
        for(i=1;i<=e;i++)
        {
        printf("Enter edges a,b and w:"); scanf("%d%d%d",&a,&b,&w);
g[a][b]=g[b][a]=w;
   }
}

void prim()
{
   int current=1,totalvisited=1,min,mincost=0; visited[current]=1;
   d[current]=0;
```

```
        while(totalvisited!=v)
        {
          for(i=1;i<=v;i++)
          {
            if(g[current][i]!=0)
            if(visited[i]==0)
            if(d[i]>g[current][i])
            {
              d[i]=g[current][i];
              p[i]=current;
                    }
                    }
                    min=32767;
                    for(i=1;i<=v;i++)
                    {
                    if(visited[i]==0)
                    if(d[i]<min)
                    {
                    min=d[i];
                    current=i;
                     }
                    }
                    visited[current]=1;
                    totalvisited++;
            }

            for(i=1;i<=v;i++)
            mincost=mincost+d[i];
            printf("Minimum cost of the spanning tree is %d",mincost);
      }
```

e) Output

Enter number of vertices : 3
Enter number of edges : 3
Enter edges a , b &w: 1 2 10
Enter edges a , b &w: 2 3 20
Enter edges a , b &w: 3 1 30

Minimum cost of the spanning tree is: 30

e) Conclusion:
Thus prim's algorithm to find minimum spanning tree is studied and implemented successfully.

VIVA QUESTIONS

EXPERIMENT NO:7 DOP: DOS: GRADE:

TITLE OF EXPERIMENT: KNAPSACK PROBLEM USING GREEDY METOHD

AIM: To find Optimal solution for Knapsack Problem by using Greedy Method.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory:

Greedy method or technique is used to solve Optimization problems. A solution that can be maximized or minimized is called Optimal Solution.

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a mass and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible. It derives its name from the problem faced by someone who is constrained by a fixed-size knapsack and must fill it with the most valuable items. The most common problem being solved is the 0-1 knapsack problem, which restricts the number x_i of copies of each kind of item to zero or one.

In Knapsack problem we are given:1) n objects 2) Knapsack with capacity m, 3) An object i is associated with profit W_i , 4) An object i is associated with profit P_i , 5) when an object i is placed in knapsack we get profit $P_i X_i$.

Here objects can be broken into pieces (X_i Values)

The Objective of Knapsack problem is to maximize the profit.

c) Algorithm:

```
Algorithm GreedyKnapsack(m,n)
//p[1:n] and w[1:n] contain the profits and wei hts respectively
//of the n objects ordered such that p[i]
//m is the knapsack size and x[1:n] is the solution vector
{
        for i := 1 to n do x[i] := 0.0; //Initialize x.
        U :=m;
        for i :=1 to n do
        {


        }
        if (i<=n) then x[i] := U/w[i];
}
```

d) Program:

```
#include<stdio.h>
#include<conio.h>

main()
{
   int n,m,i,u;
   int p[20],w[20]; float x[20];
   float optimal=0.0;
   printf("Enter number of objects:");
   scanf("%d",&n);
   printf("Enter capacity of KnapSack:"); scanf("%d",&m);
   printf("Enter profits in decreasing order of Pi/Wi:"); for(i=1;i<=n;i++)
   scanf("%d",&p[i]);
   printf("Enter Weights in decreasing order of Pi/Wi:"); for(i=1;i<=n;i++)
   scanf("%d",&w[i]);
   for(i=1;i<=n;i++)
   x[i]=0.0;
   u=m;
   for(i=1;i<=m;i++)
   {
                if(w[i]>u)
                        break;
                else
                        x[i]=1.0;
                u=u-w[i];
   }
        if(i<=n)
                x[i]=(float)u/w[i];
        printf("The x values are\n");

        for(i=1;i<=n;i++)
                printf("%f\t",x[i]);
        for(i=1;i<=n;i++)
                optimal=optimal+p[i]*x[i];
        printf("\nOptimal Solution is %f",optimal);
        getch();
}
```

e) Output:

Enter the number of objects: 3
Enter the capacity of Knapsack: 20
Enter profits in decreasing order of pi/wi : 16 15 14
Enter weights in decreasing order of pi/wi : 10 10 10

The x-values are :
1.000000 1.000000 0.000000 Optimal Solution is : 31.000000

e) Conclusion:
Thus the optimal solution for a Knapsack problem is obtained by using greedy strategy successfully.

VIVA QUESTIONS

EXPERIMENT NO: 8 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: 0/1 KNAPSACK PROBLEM

AIM: To study and implement solution of 0/1 knapsack problem using dynamic programming.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory:

A thief robbing a store and can carry a maximal weight of W into their knapsack. There are n items and i^{th} item weigh w_i and is worth v_i dollars. What items should thief take?

0/1 knapsack problem: In 0/1 knapsack problem items may not be broken into smaller pieces, so thief may decide either to take an item or to leave it (binary choice), but may not take a fraction of an item.

Dynamic-Programming Solution to the 0-1 Knapsack Problem

Let i be the highest-numbered item in an optimal solution S for W pounds. Then $S` = S - \{i\}$ is an optimal solution for $W - w_i$ pounds and the value to the solution S is V_i plus the value of the sub-problem.

We can express this fact in the following formula: define $c[i, w]$ to be the solution for items 1,2, . . . , i and maximum weight w. Then

$$c[i,w] \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ c[i-1, w] & \text{if } w_i \geq 0 \\ \max [v_i + c[i-1, w-w_i], & \text{if } i>0 \text{ and } w \geq w_i \end{cases}$$

This says that the value of the solution to i items either include i^{th} item, in which case it is v_i plus a subproblem solution for $(i - 1)$ items and the weight excluding w_i, or does not include i^{th} item, in which case it is a subproblem's solution for $(i - 1)$ items and the same weight. That is, if the thief picks item i, thief takes v_i value, and thief can choose from items$w - w_i$, and get $c[i - 1, w - w_i]$ additional value. On other hand, if thief decides not to take item i, thief can choose from item 1,2, . . . , i- 1 upto the weight limit w, and get $c[i - 1,w]$ value. The better of these two choices should be made.

The algorithm takes as input the maximum weight W, the number of items n, and the two sequences $v = <v_1, v_2, . . . , v_n>$ and $w = <w_1, w_2, . . . , w_n>$. It stores the $c[i, j]$values in the table, that is, a two dimensional array, $c[0 . . n, 0 . . w]$ whose entries are computed in a row-major order. That is, the first row of c is filled in from left to right, then the second row, and so on. At the end of the computation, $c[n, w]$ contains the maximum value that can be picked into the knapsack.

c) Algorithm:

```
0/1Knapsack(S, W)
//Input: set S of items with benefit bi and weight wi; max. weight W
//Output: benefit of best subset with weight at most W
// Sk: Set of items numbered 1 to k.
//Define B[k,w] = best selection from Sk with weight exactly equal to w
{
for w ← 0 to n-1 do
B[w] ← 0
for k ← 1 to n do
{
for w ← W downto wk do
{
if B[w-wk]+bk > B[w] then
B[w] ← B[w-wk]+bk
}
}
}
```

d) Program:

```
#include<stdio.h>
#define MAX 50
int p[MAX],w[MAX],n;
int knapsack(int,int);
int max(int,int);
main()
{
        int m,i,optsoln;
        printf("Enter no. of objects: ");
        scanf("%d",&n);
        printf("\nEnter the weights:\n");
        for(i=1;i<=n;i++)
        scanf("%d",&w[i]);
        printf("\nEnter the profits:\n");
        for(i=1;i<=n;i++)
        scanf("%d",&p[i]);
        printf("\nEnter the knapsack capacity:");
        scanf("%d",&m);
        optsoln=knapsack(1,m);
        printf("\nThe optimal soluntion is:%d",optsoln);
        getch();
}
int knapsack(int i,int m)
{
        if(i==n)
```

```
        return (w[n]>m) ? 0 : p[n];
        if(w[i]>m)
        return knapsack(i+1,m);
        return max(knapsack(i+1,m),knapsack(i+1,m-w[i])+p[i]);
    }
    int max(int a,int b)
    {
        if(a>b)
        return a;
        else
        return b;
    }
```

e) Output:

```
Enter no. of objects: 3
Enter the weights:
        100
        14
        10
Enter the profits:
        20
        18
        15
Enter the knapsack capacity: 116

The optimal solution is:   38
```

f) Conclusion:

Thus 0/1 knapsack problem of finding optimal solution is solved successfully by using dynamic programming**.**

VIVA QUESTIONS

EXPERIMENT NO: 9 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: EIGHT QUEEN PROBLEM

AIM: Study Eight Queens problem and solve by using Back Tracking Technique.

FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory:

Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking formulation.

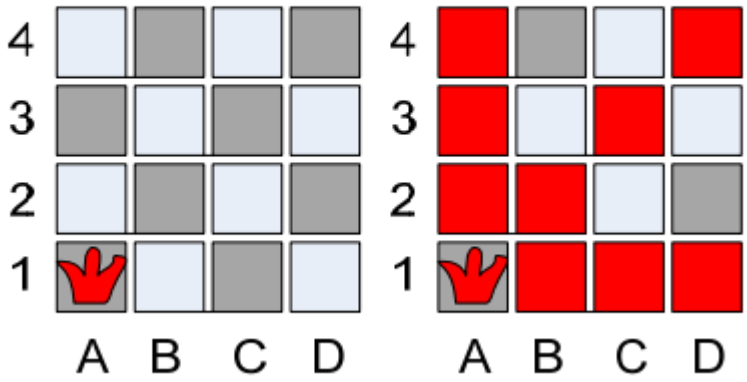
Constraints in backtracking are of 2 types:

- Explicit
- Implicit

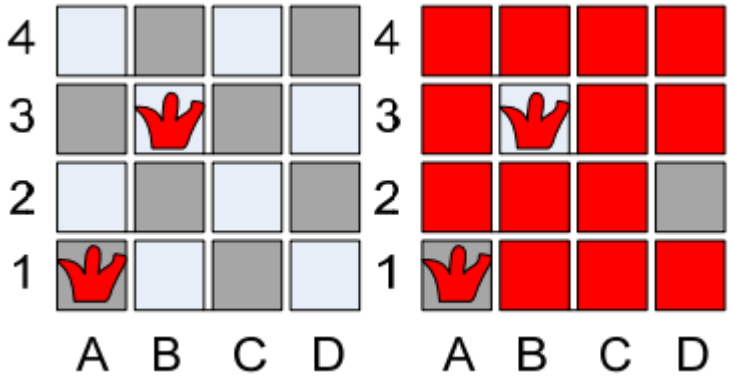
Implicit constraints: we have to avoid multiple instances of same sub-set. In backtracking desired solutions n tuple i.e (x1,x2....xn) where xi are chosen from given a finite set of Si (i.e) finding a vector that maximizes (or) minimizes a criterion function p(x1,x2...xn).
Explicit constraints: Each Xi should be taken from one of the Wi.

The eight queens puzzle is the problem of placing eight chess queens on an 8×8 chessboard so that no two queens threaten each other. Thus, a solution requires that no two queens share the same row, column, or diagonal. The eight queens puzzle is an example of the more general *n*-queens problem of placing *n* queens on an *n*×*n* chessboard, where solutions exist for all natural numbers *n*with the exception of *n*=2 or *n*=3.

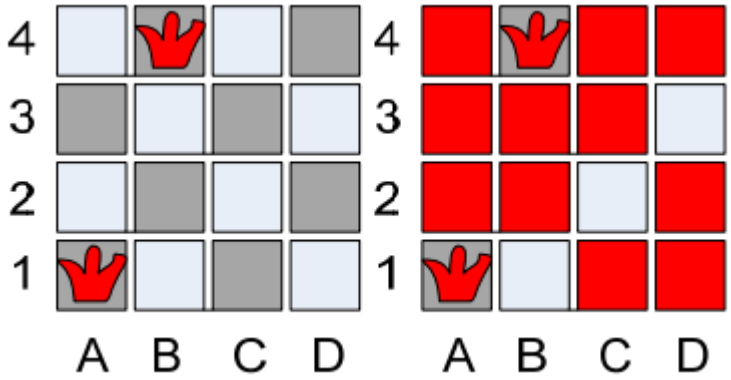
We place the first queen on A1:



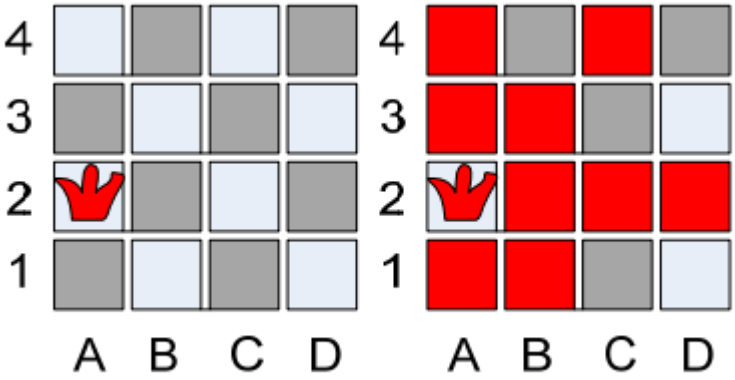
Note the positions which Q1 is attacking. So the next queen Q2 has to options: B3 or B4. We choose the firs one B3.



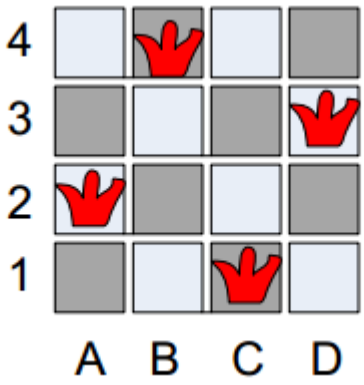
Again with red we show the prohibited positions. It turned out that we cannot place the third queen on the third column. B3 gives problem for the third queen, so there is only one position left – B4.



Now we have admissible position for Q3, but it will make impossible to place Q4 since the only place is D3. We need to go for second backtrack. Why? The reason is that there is no position for Q2, which will satisfy any position for Q4 or Q3. Hence we need to deal with the position of Q1. We have started from Q1 so we will continue upward and placing the queen at A2.



Now it is easy to see that Q2 goes to B4, Q3 goes to C1 and Q4 takes D3:



To find this solution we had to perform two backtracks. So what now? In order to find all solutions we use as you can guess – backtrack!

c) Algorithm:

```
Algorithm NQueens (k, n)
        //Using backtracking, this procedure prints all possible placements of n queens
        //on an n x n chessboard so that they are non-attacking
        {
        for i ← 1 to n do
        {
        if(Place(k,i) )
        {
        x[k] ← i
        if (k=n)
        write ( x[1...n])
        else
        Nqueens (k+1, n)
        }
        }
        }
Algorithm Place( k, i)
        //Returns true if a queen can be placed in kth row and ith column. Otherwise it
        //returns false. x[] is a global array whose first (k-1) values have been set. Abs(r)
        //returns the absolute value of r.
        {
        for j ← 1 to k-1 do
        {
        if ( (x[j]=i or Abs(x[j]-i) = Abs(j-k) )
        {
        return false
        }
        }
        }
```

d) Program:

```
#include <stdio.h>
#include<conio.h>

int row[8],s=0;
int safe(int,int);
void putboard();
void queen(int);
int safe(int x, int y)
{
   int i; for(i=1;i<=y;i++)
   if( row[y-i]==x || row[y-i]==x-i || row[y-i]==x+i)
   return 0;
   return 1;
}
void putboard()
{
   int x,y;
   printf("\nSolution # %d",++s);
   printf(":\n-----------------------------\n");
   for(y=0;y<8; y++)
   {
                    for (x=0;x<8;x++)
                              if(x==row[y])
                              printf("| Q ");
                              else
                              printf("|   ");
                    printf("|\n---------------------------\n");
          }

          getch();
}
void queen(int y)
{
          int x;
          for(x=0;x<8;x++)
          {
                    row[y-1]=x;
                    if( safe(x,y-1) )
                              if (y<8)
                              queen(y+1);
         else
         putboard();
   }
}
```

```
void main()
{
    queen(1);
}
```

e) Output:

```
Solution # 1:
-------------------------------
| Q |   |   |   |   |   |   |
-------------------------------
|   |   |   |   | Q |   |   |
-------------------------------
|   |   |   |   |   |   | Q |
-------------------------------
|   |   |   |   |   | Q |   |
-------------------------------
|   |   | Q |   |   |   |   |
-------------------------------
|   |   |   |   |   |   | Q |
-------------------------------
|   | Q |   |   |   |   |   |
-------------------------------
|   |   |   | Q |   |   |   |
-------------------------------
```

f) Conclusion:

Eight Queens Problem using Back Tracking Technique is Solved and implemented successfully.

VIVA QUESTIONS

EXPERIMENT NO: 10 **DOP:** **DOS:** **GRADE:**

TITLE OF EXPERIMENT: TRAVELLING SALESPERSON PROBLEM

AIM: To study and implement Travelling salesperson problem using dynamic programming.

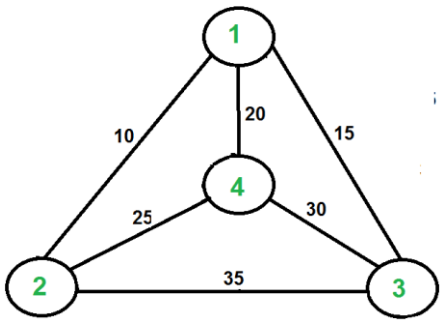
FACILITIES REQUIRED AND PROCEDURE

a) Facilities required to do the experiment:

S. No.	Facilities required	Quantity
1	System	1
2	Compiler to run C program	-

b) Theory:

Travelling Salesman Problem (TSP): Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.
Note the difference between Hamiltonian Cycle and TSP. The Hamiltoninan cycle problem is to find if there exist a tour that visits every city exactly once. Here we know that Hamiltonian Tour exists (because the graph is complete) and in fact many such tours exist, the problem is to find a minimum weight Hamiltonian Cycle.
For example, consider the graph shown in figure below. A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is 10+25+30+15 which is 80.



The problem is a famous NP hard problem. There is no polynomial time know solution for this problem.

Let the given set of vertices be {1, 2, 3, 4,….n}. Let us consider 1 as starting and ending point of output. For every other vertex i (other than 1), we find the minimum cost path with 1 as the starting point, i as the ending point and all vertices appearing exactly once. Let the cost of this path be cost(i), the cost of corresponding Cycle would be cost(i) + dist(i, 1) where dist(i, 1) is the distance from i to 1. Finally, we return the minimum of all [cost(i) + dist(i, 1)] values. This looks simple so far. Now the question is how to get cost(i)?
To calculate cost(i) using Dynamic Programming, we need to have some recursive relation in terms of sub-problems. Let us define a term *C(S, i) be the cost of the minimum cost path visiting each vertex in set S exactly once, starting at 1 and ending at i.*

We start with all subsets of size 2 and calculate C(S, i) for all subsets where S is the subset, then we calculate C(S, i) for all subsets S of size 3 and so on. Note that 1 must be present in every subset.

If size of S is 2, then S must be {1, i},
C(S, i) = dist(1, i)
Else if size of S is greater than 2.
C(S, i) = min { C(S-{i}, j) + dis(j, i)} where j belongs to S, j != i and j != 1.

For a set of size n, we consider n-2 subsets each of size n-1 such that all subsets don't have nth in them. Using the above recurrence relation, we can write dynamic programming based solution.
There are at most $O(n*2^n)$ subproblems, and each one takes linear time to solve. The total running time is therefore $O(n^2*2^n)$. The time complexity is much less than O(n!), but still exponential. Space required is also exponential. So this approach is also infeasible even for slightly higher number of vertices.

c) Algorithm:

Algorithm TSP(start city, current city,next city, path)
//**Purpose:** Tof ind the solution for TSP problem using exhaustive search
//**Input:** The start city, current city,next city and the path
//**Output:** The minimum distance covered along with the path
Step 1: Check for the disconnection between the current city and the next city
Step 2: Check whether the travelling sales person has visited all the cities
Step 3: Find the next city to be visited
Step 4: Find the solution and terminate

d) Program

```
#include<stdio.h>
int s,c[100][100],ver;
float optimum=999,sum;
/* function to swap array elements */
void swap(int v[], int i, int j)
{
   int t;
   t = v[i];
   v[i] = v[j];
   v[j] = t;
}
/* recursive function to generate permutations */
void brute_force(int v[], int n, int i)
{
   // this function generates the permutations of the array from element i to element n-
1
   int j,sum1,k;
   //if we are at the end of the array, we have one permutation
   if (i == n)
```

```
        {
        if(v[0]==s)
        {
        for (j=0; j<n; j++)
        printf ("%d ", v[j]);
        sum1=0;
        for( k=0;k<n-1;k++)
        {
        sum1=sum1+c[v[k]][v[k+1]];
        }
        sum1=sum1+c[v[n-1]][s];
        printf("sum = %d\n",sum1);
        getch();
        if (sum1<optimum)
        optimum=sum1;
        }
        }
        else
        // recursively explore the permutations starting at index i going through index n-1*/
        for (j=i; j<n; j++)
        { /* try the array with i and j switched */
        swap (v, i, j);
        brute_force (v, n, i+1);
        /* swap them back the way they were */
        swap (v, i, j);
    }
    }
    void nearest_neighbour(int ver)
    {
        int min,p,i,j,vis[20],from;
        for(i=1;i<=ver;i++)
        vis[i]=0;
        vis[s]=1;
        from=s;
        sum=0;
        for(j=1;j<ver;j++)
        {
        min=999;
        for(i=1;i<=ver;i++)
        if(vis[i] !=1 &&c[from][i]<min && c[from][i] !=0 )
        {
        min= c[from][i];
        p=i;
        }
        vis[p]=1;
        from=p;
        sum=sum+min;
```

```
        }
        sum=sum+c[from][s];
    }
    main ()
    {
        int ver,v[100],i,j;
        printf("Enter n : ");
        scanf("%d",&ver);
        for (i=0; i<ver; i++)
        v[i] = i+1;
        printf("Enter cost matrix\n");
        for(i=1;i<=ver;i++)
        for(j=1;j<=ver;j++)
        scanf("%d",&c[i][j]);
        printf("\nEnter source : ");
        scanf("%d",&s);
        brute_force (v, ver, 0);
        printf("\nOptimum solution with brute force technique is=%f\n",optimum);
        nearest_neighbour(ver);
        printf("\nSolution with nearest neighbour technique is=%f\n",sum);
        printf("The approximation val is=%f",((sum/optimum)-1)*100);
        printf(" % ");
        getch();
    }
```

e) Output:

```
Enter n:        5
Enter cost matrix:
        0 3 1 5 8
        3 0 6 7 9
        1 6 0 4 2
        5 7 4 0 3
        8 9 2 3 0

Enter source:           1

    1 2 3 4 5 sum =   24
    1 2 3 4 5 sum =   19
    1 2 3 4 5 sum =   24
    1 2 3 4 5 sum =   16
    1 2 3 4 5 sum =     20
    1 2 3 4 5 sum =     23
    1 2 3 4 5 sum =     25
    1 2 3 4 5 sum =     24
    1 2 3 4 5 sum =     29
    1 2 3 4 5 sum =     20
    1 2 3 4 5 sum =     16
    1 2 3 4 5 sum =     24
    1 2 3 4 5 sum =     32
```

```
1 2 3 4 5 sum =     32

1 2 3 4 5 sum =     28
1 2 3 4 5 sum =     24
1 2 3 4 5 sum =     24
1 2 3 4 5 sum =     19
1 2 3 4 5 sum =     24
1 2 3 4 5 sum =     28
1 2 3 4 5 sum =     24
1 2 3 4 5 sum =     25
1 2 3 4 5 sum =     29
1 2 3 4 5 sum =     32

Optimal solution with brute force technique is:  16.00000
Solution with nearest neighbor technique is      :        16.00000
The approximation value is: 0.00000%
```

f) Conclusion:

Thus traveling salesperson problem using greedy strategy is studied and implemented successfully.

VIVA QUESTIONS